

PostgreSQL: Persistencia Histórica de Métricas & pg_cron

Guía Completa: Instalación, Configuración y Captura en Tiempo Real

Esta documentación detalla la arquitectura para registrar continuamente la actividad de las sesiones en ejecución (`pg_stat_activity`) cruzando opcionalmente el tiempo de kernel por consulta con `pg_stat_kcache` mediante el programador nativo `pg_cron`.

PASO 1 Instalación del Paquete del Sistema Operativo (OS)

Instala el paquete correspondiente a la versión de PostgreSQL configurada en tu servidor:

En Debian / Ubuntu / Raspberry Pi OS:

```
# Reemplazar '16' por la versión instalada (ej. postgresql-14-cron, postgresql-15-cron)
sudo apt-get update
sudo apt-get install postgresql-16-cron
```

En RHEL / CentOS / Rocky Linux / AlmaLinux:

```
# Reemplazar '16' por la versión de PostgreSQL
sudo dnf install pg_cron_16
```

PASO 2 Configuración del Parámetro en postgresql.conf

Edita `postgresql.conf` para cargar las librerías compartidas necesarias:

```
# Cargar extensiones necesarias
shared_preload_libraries = 'pg_stat_statements, pg_stat_kcache, pg_cron'

# Base de datos que administrará el daemon de cron
cron.database_name = 'postgres'
```

Reinicio Requerido del Servicio:

Aplica los cambios reiniciando el servicio:

```
sudo systemctl restart postgresql
```

PASO 3 Creación de Extensión y Tabla de Historial

Ejecuta el DDL en la base de datos para habilitar la extensión y la tabla de destino:

```
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Tabla para el historial continuo de snapshots
CREATE TABLE IF NOT EXISTS pg_stat_activity_history (
    snapshot_time timestamp with time zone DEFAULT now(),
    pid integer,
    username text,
    datname text,
    state text,
    wait_event_type text,
    wait_event text,
    cpu_user_seconds numeric,
    cpu_system_seconds numeric,
    query_id bigint,
    query text
);

-- Índice para acelerar el filtrado por rangos de tiempo
CREATE INDEX IF NOT EXISTS idx_stat_history_time ON pg_stat_activity_history (snapshot_time);
```

PASO 4 Programación de Jobs con pg_cron

Aclaración sobre la vista pg_stat_kcache_detail:

La vista `pg_stat_kcache_detail` acumula estadísticas globales agrupadas por texto de consulta y usuario (`datname`, `rolname`), desagregadas en fase de planificación (`plan_user_time`) y ejecución (`exec_user_time`). Dado que `pg_stat_activity` expone el estado de los procesos en tiempo real por PID, las métricas de CPU acumuladas de kcache se integran mediante cruce por `query` o `query_id` desde `pg_stat_statements`.

1. Snapshots Cada Minuto (Captura en Tiempo Real)

Captura los procesos activos y calcula el uso total de tiempo de CPU (planificación + ejecución) acumulado:

```
SELECT cron.schedule(
    'snapshot_pg_stat_history',
    '* * * * *',
    $$
    INSERT INTO pg_stat_activity_history (
        snapshot_time, pid, username, datname, state,
        wait_event_type, wait_event, cpu_user_seconds,
        cpu_system_seconds, query_id, query
    )
    SELECT
        now(),
        a.pid,
        a.username,
        a.datname,
        a.state,
        COALESCE(a.wait_event_type, 'CPU'),
        COALESCE(a.wait_event, 'CPU Executing'),
        ROUND(CAST(COALESCE(k.plan_user_time + k.exec_user_time, 0) AS numeric), 2),
        ROUND(CAST(COALESCE(k.plan_system_time + k.exec_system_time, 0) AS numeric), 2),
        a.query_id,
        a.query
    FROM pg_stat_activity a
    LEFT JOIN pg_stat_statements s ON a.query_id = s.queryid
    LEFT JOIN pg_stat_kcache_detail k ON s.query = k.query AND a.datname = k.datname
    WHERE a.state = 'active'
        AND a.pid != pg_backend_pid();
    $$
);
```

2. Purga y Rotación de Datos (Diario a las 03:00 AM)

Elimina automáticamente registros antiguos de más de 30 días:

```
SELECT cron.schedule(
    'purga_historial_pg_stat',
    '0 3 * * *',
    $$ DELETE FROM pg_stat_activity_history WHERE snapshot_time < now() - INTERVAL '30 days'; $$
);
```

PASO 5 Verificación y Gestión de Jobs

Comandos SQL para administrar las tareas programadas:

```
-- Consultar la lista de jobs activos
SELECT jobid, schedule, command, active FROM cron.job;

-- Eliminar un job según su ID
SELECT cron.unschedule(1);
```